Working with URLs

URL and ParsedURL

Functions for working with URLs.

Building URLs

Constructing URLs dynamically is a key requirement of most web applications but JavaScript offers little help out of the box but `Combine`, `hashToQuery` and `formURLEncode` make simple work of it. `hashToQuery` makes the querystring, calling `formURLEncode` to processes any special characters in the querystring, and `combine` puts the path parts, file name and querystring together into a usable whole.

Unpeeling URLs

Of course the reverse is just as true, that web apps need to take information from the URLs, which is where `queryToHash`, `formURLDecode` and `parseURL` do the lifting. When more sophisticated tasks come up, you can call `parseURLComponents` to get an object containing all the pieces of a URL. ParsedURL contains properties for each part of a URL including the entire URL, the protocol, the domain and subdomain, the base (protocol, host and port), the path as string or array of parts, the file, the path and file, the fragment and the querystring as string or object with a property for each name/value pair.

File URLs

This namespace also has two functions that deal with the file protocol specifically: `ensureFileProtocol` and `isFile`. The latter tells you if a URL is using the file protocol and the former forces a URL to use it.

Building a URL

```
/*
  buildURL expects:
  - parts: (required) a hash with four elements, and
  - forceFile: (optional) boolean to force use of file:///

  parts hash elements:
  - buDomain: string similar to http://example.com or file:///
  - buPath: (optional) string holding the path
  - buFile: string holding the file, including extension
  - buQuery: (optional) hash of the elements to use as query string
                  OR string starting with # to use as a URL fragment
*/
function buildURL(parts, forceFile) {
    if (typeof forceFile != "boolean")
    {
        throw new Exception("buildURL was handed something of type " + (typeof forceFile) + "
        ↵ rather than a boolean for forceFile");
    }
    if (typeof parts != "object")
    {
        throw new Exception("buildURL was handed something of type " + (typeof parts) + "
        ↵ rather than an object for parts");
    }

    // do we need to force the file:// protocol
    var dmn = (forceFile && isFile(parts['buDomain'])) ?
    ↵ Jaxer.Utils.URL.ensureFileProtocol(parts['buDomain']) : parts['buDomain'];

    // fragment or querystring?
    var last = (Jaxer.Utils.String.startsWith(parts['buQuery'], '#')) ? parts['buQuery'] :
    ↵ Jaxer.Utils.URL.hashToQuery(parts['buQuery']);

    // put everything together and send it back
    return Jaxer.Utils.URL.combine(parts['buDomain'], parts['buPath'], parts['buFile'],
    ↵ parts['buQuery']);
}

var newParts = {};
var buq = {};
buq['jello'] = 'cherry';
buq['pudding'] = 'vanilla';
newParts['buDomain'] = 'http://example.com';
newParts['buPath'] = '/foo';
newParts['buFile'] = 'bar.html';
newParts['buQuery'] = buq;

var newURL = buildURL(newParts);
```

Examining a URL

```
var qry1 = 'http://movienight.example.com:8000/foo/bar/baz.php?
↵ id=001&choice;=Godfather&blueray;=true';
var qry2 = 'http://movienight.example.com:8000/foo/bar/baz.php?
↵ id=002&choice;=Animal%20House&blueray;=false';
```

Contents

API Links

Jaxer.Util.URL
Jaxer.Util.ParsedURL
Jaxer.NetworkUtils

```
var qry3 = 'file:///home/projects/movienight/poll.html#explanation';

function unPeelURL(qry) {
    var qth, purl, purlc, pathparts, path, pathFile, port, host, hostPort, fragment, file,
    ↵ base, domain, protocol, queryparts, subdomain, url;

    // grab all the possible parts
    purl = Jaxer.Utils.URL.parseURL(qry);
    pathparts = purl['pathParts'];
    path = purl['path'];
    pathFile = purl['pathAndFile'];
    port = purl['port'];
    host = purl['host'];
    hostPort = purl['hostAndPort'];
    fragment = purl['fragment'];
    file = purl['file'];
    base = purl['base'];
    domain = purl['domain'];
    protocol = purl['protocol'];
    queryparts = purl['queryParts'];
    query = purl['query'];
    qth = Jaxer.Utils.URL.queryToHash(query);
    subdomain = purl['subdomain'];
    url = purl['subdomain'];
    choice = Jaxer.Utils.URL.queryparts['choice'];
    blueray = Jaxer.Utils.URL.queryparts['blueray'] ? ' on Blueray':'';
    purlc = Jaxer.Utils.ParsedURL.parseURLComponents(hostPort, pathAndFile+fragment+query,
    ↵ protocol);

    var msg =
        'Parameter query was: ' + qry +
        'Protocol: ' + protocol +
        'Domain: ' + domain +
        'Subdomain: ' + subdomain +
        'Port: ' + port +
        'Host and Port: ' + hostPort +
        'Path Parts: ' + pathparts.toString() +
        'Path: ' + path +
        'File: ' + file +
        'Path and File: ' + pathFile +
        'Fragment: ' + fragment +
        'Query: ' + query +
        'Query Parts: ' + queryparts.toString() +
        'Query to Hash: ' + qth.toString() +
        'Whole URL: ' + url +
        'Parsed URL Components: ' + purlc.toString();

    alert(msg);

    if(choice != '') {
        alert('Movie Nights are Fun\n' + choice + ' is a cool movie to watch' + blueray);
    }
}

unPeelURL(qry1);
unPeelURL(qry2);
unPeelURL(qry3);
```

XPCOM Network utility functions

- **fixupUri:** invokes the XPCOM nsIURIFixup method and return a 'fixed' URI
- **onStartRequest:** a stub function to be overridden
- **onStopRequest:** invokes the callback function for completed requests
- **QueryInterface:** returns a QueryInterface for the provided XPCOM Interface ID
- **validateURI:** validates the provided URI using XPCOM and returns JSLib.Ok if successful